SIC 8 · BIG DATA ENGINEERING · PROJECT 03 · PROJECT PROPOSAL

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Team of 3 · 6 build days · Hybrid delivery: local Docker (generator, Kafka, Airbyte, Airflow, FastAPI, React) + cloud (Amazon S3, Snowflake)

Problem

Fleet managers cannot combine vehicle diagnostics, position and cargo temperature in one place, so route deviations, refrigeration breaches and excessive idling are hard to investigate consistently.

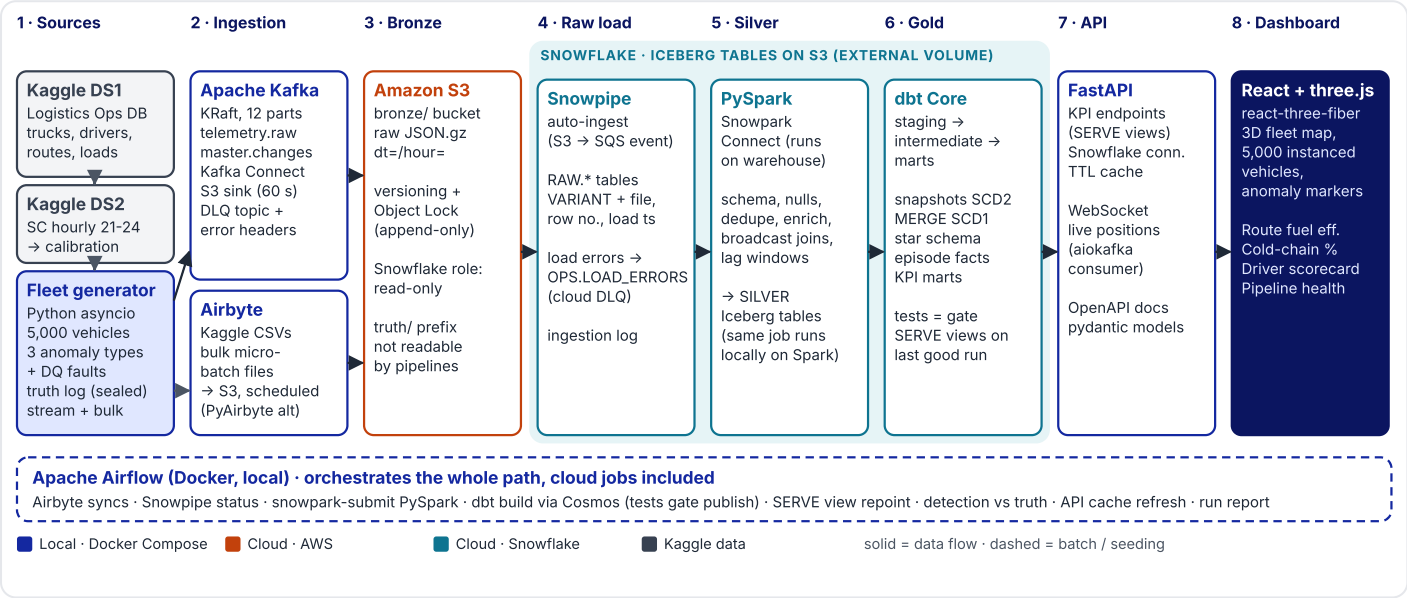
Objective

Build a telematics lakehouse and star-schema warehouse for **5,000 simulated delivery vehicles** on fixed corridors, serving route fuel efficiency, cold-chain compliance % and driver safety/idling scorecards in a 3D web dashboard.

Measurable targets

5,000 distinct vehicles · 0 duplicate keys in silver · Kafka = S3 = RAW + load errors · detection recall ≥ 95% / precision ≥ 90% vs injected truth · dashboard = API = SQL ±0.1% · event → KPI ≤ 15 min.

End-to-end flow



Data work: how the Kaggle data and the generator connect

Source	Role
DS1 Logistics Ops DB (2022–24)	**Master data.** Seeds dim_vehicle, dim_driver, dim_route, dim_facility, dim_cargo; DS1 routes become fixed corridors; DS1 loads become trips; fuel purchases benchmark simulated km/L.
DS2 Supply-chain hourly (2021–24)	**Calibration + history.** Fits hour-of-day speed, fuel-rate, temperature-noise and driver-profile distributions into `calibration.json`; kept as a historical context fact.
Generator (asyncio + Faker)	Expands DS1 to **5,000 vehicles** (`VH-00001…` keeps `source_truck_id`); emits GPS, OBD-II fuel rate, RPM, engine temp and reefer temp per ping; stream to Kafka + hourly bulk files for Airbyte; seeded config.

Anomaly	Injected → detected (rules never read labels)
Cold-chain breach	Reefer +4–10 °C for 15–90 min → temp > setpoint + 3 °C for ≥ 15 min
Excessive idling	Engine on, stopped 12–60 min → speed < 2, RPM 500–1000 ≥ 10 min, off-facility
Route deviation	3–15 km off corridor → > 2 km from polyline for ≥ 3 pings
DQ faults	Malformed, nulls, duplicates, late → Kafka DLQ / load errors, flags, dedupe, MERGE

Injections go to a sealed S3 truth prefix that pipelines cannot read; a harness scores detections against it. Units converted to metric in dbt staging.

KPIs (documented definitions)

KPI	Definition
Fuel efficiency / route	Σ km ÷ Σ litres per route (km/L); km = haversine between pings, litres = fuel_rate × Δt
Cold-chain compliance	Reefer minutes within setpoint −2/+3 °C ÷ reefer-active minutes on cold loads (simulation threshold, not certification)
Driver scorecard	Idle % = idle min ÷ engine-on min; safety score = 100 − (5·harsh/100 km + 3·speeding/100 km + 0.5·idle %), 0–100

Design by layer

Ingestion & bronze

- Kafka topics `telemetry.raw` (12 partitions by vehicle_id), `master.changes`; Kafka Connect S3 sink with native DLQ topic.
- Airbyte lands Kaggle files and bulk micro-batches on a schedule.
- S3 bronze: raw JSON.gz by dt/hour, versioning + Object Lock. Snowpipe auto-ingests into RAW; load errors = cloud DLQ.

PySpark silver + dbt gold

- PySpark job on **Snowpark Connect** (same script on local Spark): schema, nulls, dedupe, broadcast enrichment, lag windows → SILVER Iceberg.
- dbt: staging → intermediate → marts; snapshots SCD2, MERGE SCD1; star schema on Snowflake-managed Iceberg tables.
- dbt tests gate publish: SERVE views point to the last green run.

API & dashboard

- FastAPI: KPI endpoints on SERVE views with TTL cache; WebSocket of live positions from Kafka.
- React + three.js (react-three-fiber): 3D fleet map with 5,000 instanced vehicles and anomaly markers.
- Pages: fuel efficiency, cold-chain, driver scorecard, pipeline health.

Tools and alternatives

Layer	Chosen	Runs	Alternatives
Generator	Python 3.11 asyncio, aiokafka, Faker, numpy	Local	threading/multiprocessing, Mimesis, SDV
Streaming	Apache Kafka (KRaft) + Kafka Connect S3 sink	Local	Redpanda, Confluent Cloud, Amazon MSK
Batch ingestion	Airbyte OSS / PyAirbyte	Local	Airbyte Cloud, Apache NiFi, Fivetran
Bronze	Amazon S3 (versioning, Object Lock)	AWS	MinIO, ADLS, GCS
Raw load	Snowpipe auto-ingest	Snowflake	COPY INTO, external tables
Processing	PySpark on Snowpark Connect	Snowflake	Snowpark Python, EMR / Databricks Spark
Modeling	dbt Core + dbt-snowflake, dbt_utils, dbt_expectations	Snowflake	dbt Cloud, Dynamic Tables, SQLMesh
Lakehouse / SQL	Snowflake + Apache Iceberg tables on S3	Snowflake	Native Snowflake tables, Trino/Athena on Iceberg
Quality	dbt tests + contracts	Snowflake	Great Expectations, Soda Core
Orchestration	Apache Airflow + astronomer-cosmos	Local	MWAA, Dagster, Prefect
API / dashboard	FastAPI; React + Vite + TS, three.js (r3f), Recharts	Local	Flask; deck.gl / Mapbox; Superset as fallback
Infra / dev	Docker Compose, Terraform (AWS + Snowflake), GitHub CI	Local	AWS CDK, schemachange, GitLab CI

Work distribution

Member A · Data & Ingestion

Kaggle profiling, data contract, DS2 calibration, corridors, 5,000-vehicle generator, anomalies + truth log, Kafka + Connect S3 sink + DLQ, Airbyte, S3 bronze, Snowpipe + load errors, ingestion log, detection-vs-truth harness.

Member B · Snowflake, PySpark & dbt

Snowflake roles, warehouses, external volume, PySpark silver job, dbt staging/marts, snapshots SCD2, SCD1, episode detection (independent of A), KPI marts, tests, SERVE views, Iceberg features, tuning.

Member C · Platform, API & Dashboard

Repo/CI, Docker Compose, Terraform for AWS, Airflow DAGs (Airbyte, Snowflake, Cosmos), FastAPI backend, React + three.js dashboard, KPI reconciliation, README, evidence, demo video, deck lead.

6-day timeline (handbook readiness gates)

Day 1 · Gate 1
Profile data, contract, ERD + KPIs, Snowflake + S3 set-up, Docker stack, app scaffolds. *Exit:* event reaches S3.

Day 2 · Gate 2
Generator at 5,000; Snowpipe → RAW; PySpark silver v1; dbt dims; API live. *Exit:* event traced to API.

Day 3 · Core I
Anomalies + DQ faults; full silver; snapshots; first facts; pipeline DAG; live 3D map.

Day 4 · Gate 3
Episode facts, scorecards, KPI marts, tests + SERVE gate, all pages. *Exit:* raw → dashboard.

Day 5 · Gate 4
17 tests: failures, dupes, SCD, time travel, pruning, reconciliation, parity, fresh clone.

Day 6 · Gate 5 prep
Freeze 12:00, final full run, evidence, deck, recorded fallback, 2 rehearsals.

Handbook fit: PySpark (Pillar 3) and Airflow (Pillar 5) are required, so both stay; Snowflake is a handbook-listed cloud engine and its Iceberg tables cover Pillar 4. Daily 17:00 integration; every PR reviewed; XS warehouses with auto-suspend and a resource monitor cap credits.